



14. Desarrollo de IAs avanzadas con Behavior Graph

Física e Inteligencia Artificial para Videojuegos

Doble Grado en Física e Inteligencia Artificial

Curso 2025 – 2026

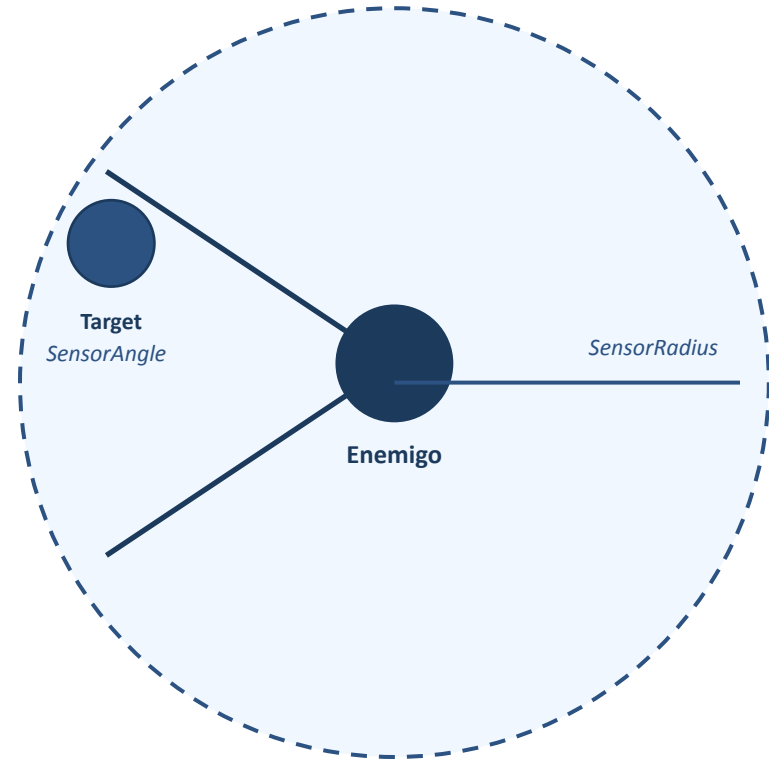
Fernando Madrid Recio

Visión a modo de cono: SensorSystem

- Componente MonoBehaviour adjunto al enemigo.
- Detecta al jugador (u objetivo) en el mundo 3D aplicando tres filtros secuenciales.
- Solo retorna el objetivo si pasa **3 filtros**. En caso contrario devuelve null.
- Filtros:
 - Target en zona (OverlapSphere)
 - Target dentro de cono (SensorAngle)
 - Sin obstáculo entre Target y yo

Parámetros configurables desde el Inspector:

- SensorRadius — radio de la esfera de detección
- SensorAngle — ángulo del campo de visión (cono)
- whatIsTarget — LayerMask del objetivo
- whatIsObstacle — LayerMask de obstáculos



Los 3 filtros de detección

1

Objetivo en zona [*Physics.OverlapSphere*]

Lanza una esfera de radio `SensorRadius`. Si no hay ningún collider con la `LayerMask` `whatIsTarget` dentro, retorna null.

2

Dentro del ángulo [*Vector3.Angle*]

Calcula el ángulo entre la dirección frontal del enemigo (`transform.forward`) y la dirección al objetivo. Si **supera** `SensorAngle / 2`, el objetivo queda **fuera** del cono de visión.

3

Sin obstáculo [*Physics.Raycast*]

Lanza un Raycast desde el enemigo hacia el objetivo con la `LayerMask` `whatIsObstacle`. Si el rayo impacta, hay un **obstáculo entre ambos** y se descarta el objetivo.

SearchTarget()

El método recorre los tres filtros en orden. Si cualquiera falla, retorna null.

```
public GameObject SearchTarget()
{
    // FILTRO 1: ¿hay algo en mi radio?
    Collider[] results = Physics.OverlapSphere(transform.position, SensorRadius, whatIsTarget);
    if (results.Length <= 0) return null;

    // FILTRO 2: ¿está dentro de mi ángulo de visión?
    Vector3 dirToTarget = (results[0].transform.position - transform.position);
    if (!(Vector3.Angle(transform.forward, dirToTarget) <= SensorAngle / 2)) return null;

    // FILTRO 3: ¿hay un obstáculo entre el objetivo y yo?
    if (!Physics.Raycast(transform.position, dirToTarget, dirToTarget.magnitude, whatIsObstacle))
    {
        return results[0].gameObject;    // ✓ Objetivo válido
    }
    return null;
}
```

Behavior Graph en Unity

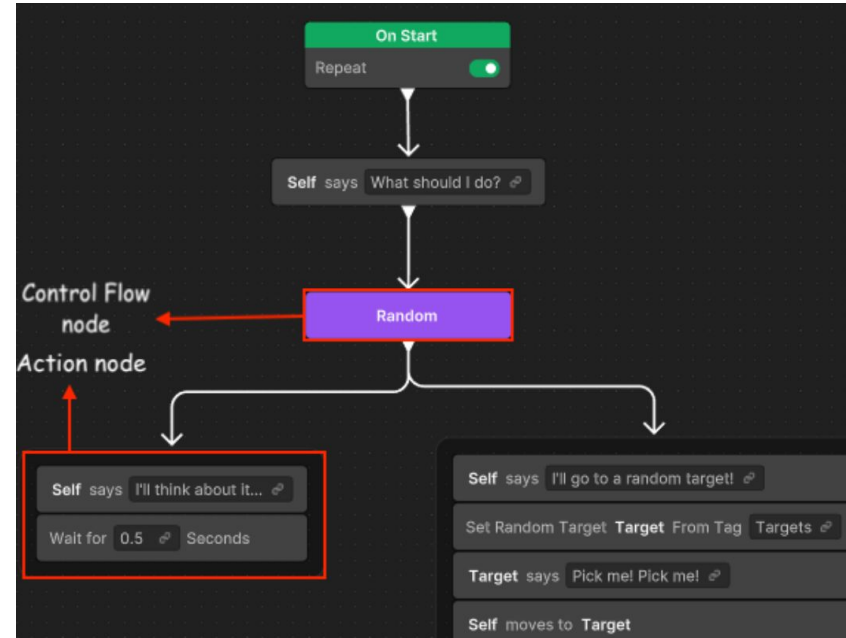
Herramienta visual de Unity (*paquete com.unity.behavior*) para definir la IA de agentes mediante nodos conectados.

¿Para qué sirve?

Sustituye la lógica de IA escrita manualmente en Update() por un grafo reusable y legible visualmente.

Ventajas clave:

- Sin código de pegamento: cada nodo es una **Action** o **Condition** autocontenida.
- **Blackboard**: Lugar donde se definen variables compartidas entre todos los nodos del grafo.
- **Extensible**: se pueden crear Actions propias en C#.
- **Depuración visual**: el grafo muestra en tiempo real qué nodo está activo.



Tipos de nodos más comunes

Action

Unidad de trabajo que ocurre en **tres fases**:

OnStart(), OnUpdate(), OnEnd()

Retorna **Running**, **Success** o **Failure**, entre otros.



Sequence

Nodo de flujo o **Flow**.
Ejecuta sus hijos en orden.
Si uno falla, la secuencia falla y se intenta repetir.
Si todos tienen éxito, la secuencia tiene éxito.



Run In Parallel

Nodo de flujo o **Flow**.
Ejecuta todos sus hijos **simultáneamente**.
Útil para combinar movimiento y búsqueda de target al mismo tiempo.



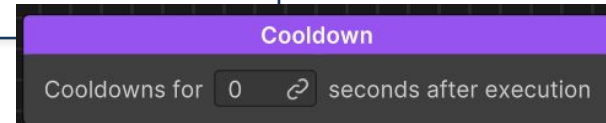
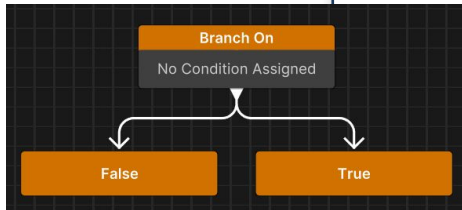
Condicionales (en naranja)

Controla el flujo del grafo: switches, condiciones (If), branches (if-else)



Modifier

Modifica el resultado de la acción (o acciones) en donde se “acopla”.
Por ejemplo, un **Cooldown**



Custom Actions

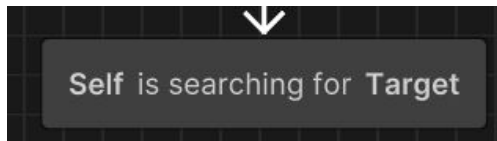
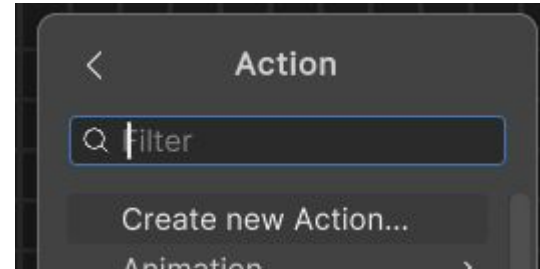
El Behavior Graph permite crear acciones **propias** o **personalizadas**.

Esto nos da una gran libertad, ya que podemos desarrollar comportamientos complejos e integrarlos directamente en nuestro árbol.

Para crear una Action propia, basta con dar al **espacio** dentro del Behavior Graph > Create new action...

También pedirá que desarrollemos una “historia” para nuestra acción.

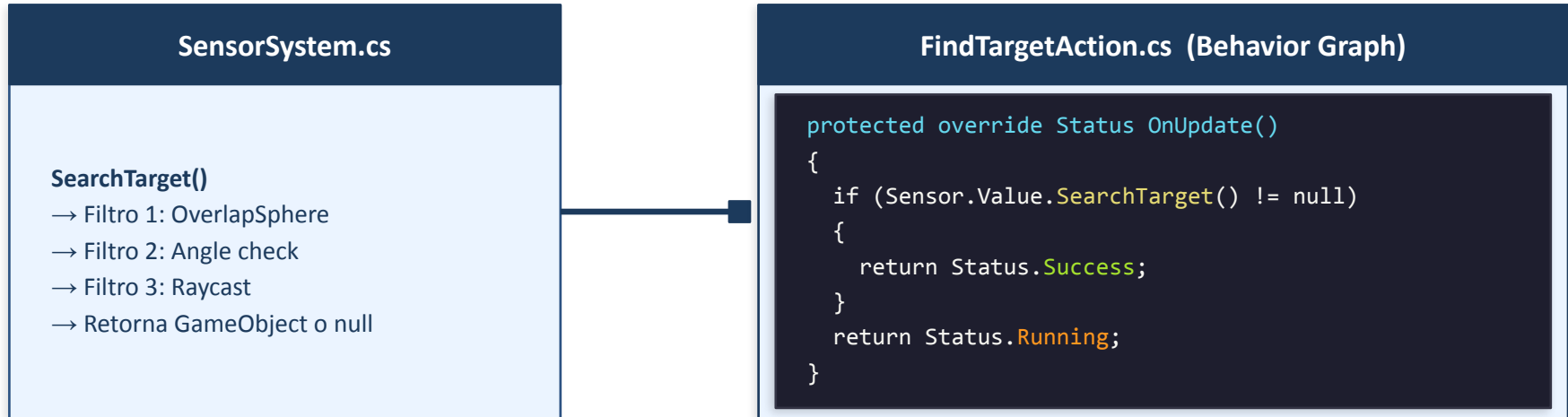
Esta historia determinará que datos o variables quedan expuestas en la representación **visual** de la acción.



Conexión: SensorSystem ↔ Behavior Graph

En nuestro caso, nos interesa **integrar** nuestro SensorSystem en el árbol.

Para ello, ejecutaremos nuestro método **SearchTarget()** durante el OnUpdate() de la tarea, devolviendo **Success/Running** en función del resultado.



La Action sólo decide qué Status devolver. Toda la lógica de detección vive en SensorSystem. El Behavior Graph **consume datos** del SensorSystem a través del Blackboard.

Custom Action: FindTargetAction

```
[Serializable, GeneratePropertyBag]
[NodeDescription(name: "FindTarget", story: "[Sensor] is searching for [Target]", category: "Action")]
public partial class FindTargetAction : Action
{
    // Variable del Blackboard que apunta al SensorSystem del enemigo
    [SerializeReference] public BlackboardVariable<SensorSystem> Sensor;
    [SerializeReference] public BlackboardVariable<SensorSystem> Target;
    protected override Status OnUpdate()
    {
        if (Sensor.Value.SearchTarget() != null)
        {
            Target.Value = Sensor.Value.SearchTarget(); // Se establece en el Blackboard el target encontrado.
            return Status.Success; // Objetivo encontrado → nodo exitoso
        }
        return Status.Running; // Seguimos buscando
    }
}
```